

1. Fondamentaux des tests

Qu'est-ce que le test?

Le test de logiciels permet d'évaluer la qualité des logiciels et de réduire le risque de défaillance en cours d'utilisation.

C'est un ensemble d'activités visant à découvrir les défauts et à évaluer la qualité des artefacts logiciels

Le test peut être requis pour répondre à des **exigences contractuelles ou légales**, ou pour se conformer à **des standards réglementaires**.

#Vérification, c'est-à-dire le contrôle de la conformité du système aux **exigences** spécifiées (**Testeur**)

#Validation, c'est-à-dire le contrôle de la conformité du système aux **besoins des utilisateurs** et des autres parties prenantes dans son environnement opérationnel. (**PO/ Client**)

Tests dynamiques : sont réalisés sur un logiciel en cours d'exécution. Implique l'exécution du logiciel

Tests statiques : effectués sur du code source non exécuté et sur de la documentation. Comprend les revues

#Objectifs du test :

- Évaluer les produits d'activités tels que les exigences, les User Stories, les conceptions et le code.
- Provoquer des défaillances et trouver des défauts.
- Assurer la couverture requise d'un objet de test.
- Réduire le niveau de risque d'une qualité logicielle insuffisante.
- Vérifier si les exigences spécifiées ont été satisfaites.
- Vérifier qu'un objet de test est conforme aux exigences contractuelles, légales et réglementaires.
- Fournir des informations aux parties prenantes pour leur permettre de prendre des décisions éclairées.
- Construire la confiance dans la qualité de l'objet de test.
- Valider si l'objet de test est complet et fonctionne comme attendu par les parties prenantes.

#Débogage : est l'activité de développement qui trouve, analyse et corrige (élimine) de tels défauts (Cause des défaillances).

Le **débogage** implique les tâches suivantes:

- Reproduction d'une défaillance.
- Diagnostic (trouver la cause racine).
- Correction de la cause.

#Test de confirmation : vérifier si les correctifs apportés ont résolu le problème (**échec → succès**)

#Tests de régression : vérifier si les corrections entraînent des défaillances dans d'autres parties de l'objet test (**succès → succès**)

Test et assurance qualité

QC : Le test est une forme de contrôle de la qualité qui est une approche **corrective axée sur le produit**

QA : L'assurance qualité est une approche **préventive axée sur les processus**, qui se concentre sur la mise en œuvre et l'amélioration des processus. Elle part du principe que si un bon processus est suivi correctement, il générera un bon produit.

Erreurs, défauts, défaillances et causes racine

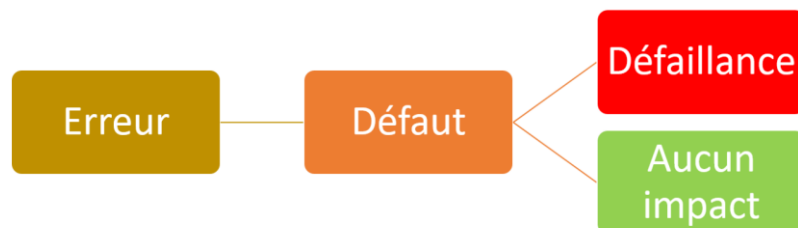
Erreur : Action humaine produisant un résultat incorrect (généralement imprévue)

Défaut (Bug/fautes): Une imperfection dans un composant ou un système qui peut conduire à ce qu'un composant ou un système n'exécute pas les fonctions requises, peuvent se trouver **dans la documentation, comme une spécification des exigences ou un script de test, dans le code source ou dans un artefact de support tel qu'un fichier de build**

Défaillances : Écart constaté du composant ou système par rapport au livrable, au service ou au résultat attendu. Incapacité d'un système ou d'un composant d'exécuter une fonction requise dans les limites spécifiées (Si un défaut dans le code est exécuté).

Une cause racine est une raison fondamentale de l'apparition d'un problème

Anomalie : Toute condition qui dévie des attentes basées sur les **exigences de spécifications, documents de conception, documents utilisateurs, standards** etc. (**statique**)



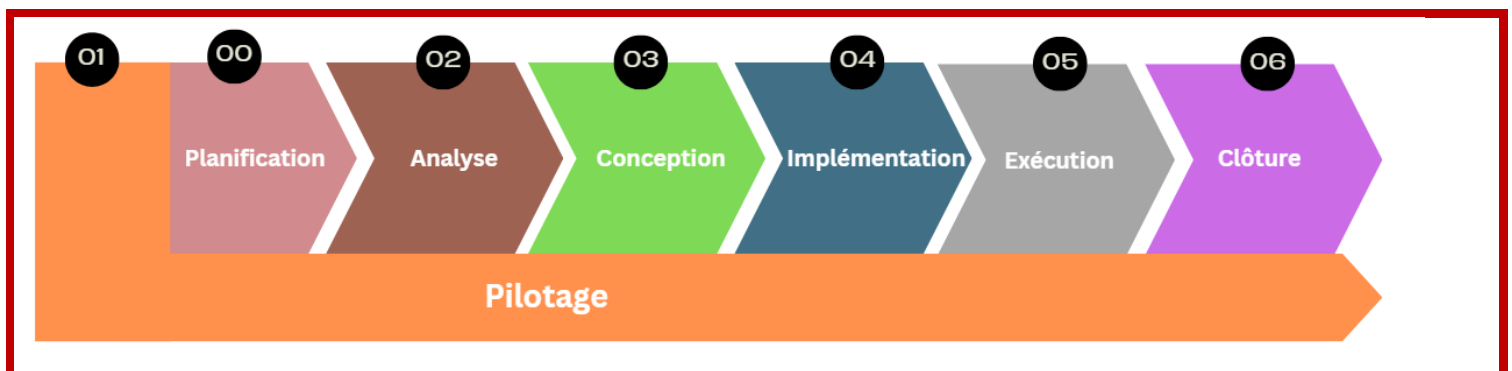
Faux-positif : Un résultat de test dans lequel un défaut est rapporté alors qu'en réalité ce défaut n'existe pas dans l'objet de test (état) **NOK → OK** (Réellement)

Faux-négatif : Un résultat de test qui n'a pas identifié la présence d'un défaut qui est réellement présent dans l'objet de test (état) **OK → NOK** (Réellement)

Les principes généraux des tests

- 1 : Les tests montrent la présence de défauts
- 2 : Les tests exhaustifs sont impossibles
- 3 : Tester tôt
- 4 : Regroupement des défauts (principe de Pareto 80% des défauts se regroupe dans 20% de système)
- 5 : #Usure des tests = paradoxe du pesticide (appliquer les mêmes tests devient inefficace de détecter les défauts) à répétition des mêmes tests peut avoir un effet **bénéfique**, par exemple dans les tests de régression automatisés
- 6 : Les tests dépendent du contexte (tester une app médicale est $\neq$ aux app e-commerce...)
- 7 : L'illusion de l'absence des défauts/défaillances

Processus de test fondamental



T0 : planification



Définir les objectifs du test, puis à sélectionner une approche qui permette d'atteindre au mieux les objectifs **un planning doit organiser** et distribuer l'effort de test dans le temps.

Etablir la Config logiciel GCL

Phase de Planification des Tests

1. **Définition des Objectifs de Test** : Identification des objectifs globaux que les tests doivent atteindre.
2. **Développement de la Stratégie de Test** : Élaboration d'une approche pour atteindre les objectifs de test, y compris la sélection des techniques de test, des outils et des niveaux de test.
3. **Planification des Ressources** : Identification des ressources humaines, matérielles et logicielles nécessaires pour les tests.
4. **Estimation de l'Effort de Test** : Évaluation du temps et des efforts requis pour les différentes activités de test.
5. **Développement du Calendrier des Tests** : Création d'un calendrier pour toutes les activités de test, y compris les jalons importants.
6. **Détermination de l'Environnement de Test** : Définition et préparation de l'infrastructure nécessaire pour réaliser les tests.
7. **Sélection des Outils de Test** : Identification et préparation des outils de test nécessaires.
8. **Définition des Critères d'Entrée et de Sortie** : Spécification des conditions qui doivent être remplies pour commencer et terminer une phase ou une activité de test.

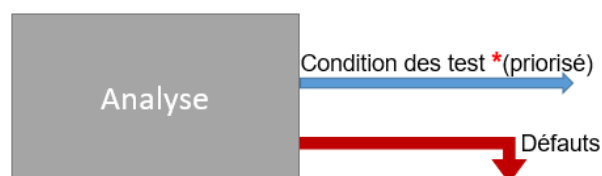
T1 : pilotage



Le pilotage des tests implique la vérification continue de toutes les activités de test et la comparaison des progrès réels par rapport au plan se base sur l'évaluation des critères de sortie **DOD**

Indicateur de test ***(test coverage, exp nb bug détecté)**

T2 « Quoi tester »

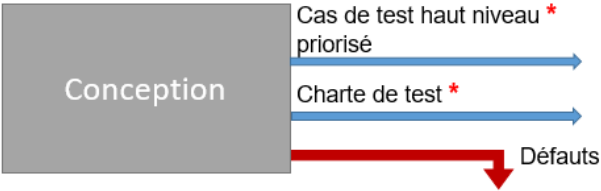
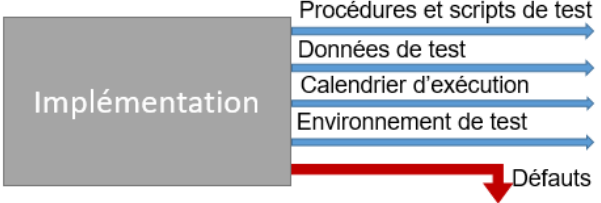
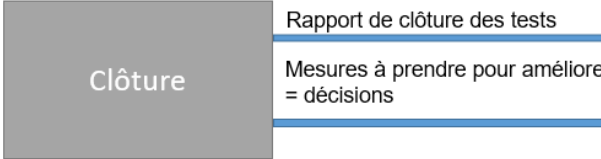


C'est l'activité qui permet au testeur de :

1. lire apprendre et vérifier la qualité des spec
2. Identifier la liste des éléments à tester ou la liste des **conditions de test***, et les prioriser ainsi que les risques et les niveaux de risque correspondants
3. Capturer **la traçabilité bidirectionnelle** entre chaque élément des bases de test et les conditions de test associées, → **construire un patrimoine de test**

T3 « Comment tester »

1. définition des exigences en matière de données de test
2. la **conception de l'env de test et l'identification de toute autre infrastructure et de tout autre outil nécessaires**
3. structurer les tests grâce au concept de la **traçabilité bidirectionnelle*** càd que chaque exigence est liée aux

	cas de tests qui la vérifient, et que chaque cas de test est lié aux exigences qu'il couvre. → une bonne couverture de test
T4 « est-ce que tout est en place pour exécuter les tests ? »  Procédure de test : Test manuel Script de test : test automatisé	! Création du testware* nécessaire à l'exécution des tests 1. Développer et prioriser les procédures de test 2. Créer des scripts de test automatisés 3. Créer des suites de tests à partir des procédures 4. Construire l'env de test (harnais*, virtualisation) 5. Préparer les données de test 6. Vérifier et mettre à jour la traçabilité bidirectionnelle entre les bases de test, les conditions de test, les cas de test, les procédures de test et les suites de tests.
T5 «Finalement, je peux dérouler les tests» Un log de test est un rapport qui résume les activités de test effectuées et les résultats obtenus pour une application logicielle spécifique 	1. Exécuter les tests manuels et automatisés 2. Comparer les résultats attendus et obtenus 3. Analyser les anomalies afin d'établir leurs causes probables 4. Signaler les défauts sur la base des défaillances observées, 5. Enregistrer les résultats de l'exécution des tests (par ex. : OK, NOK, Bloqué...), 6. Vérifier et mettre à jour la traçabilité bidirectionnelle entre les bases de test, les conditions de test, les cas de test, les procédures de test et les résultats des tests.
T6 Clôture des tests « une phase de prise de recul »  Un bilan de test → informel Un PV (procès-verbal) → formel	→ Campagne réussite ou ratée ? 1. Vérifier si tous les rapports de défauts sont clôturés 2. Créer un rapport de synthèse de test 3. Finaliser et archiver l'environnement de test, les données de test, l'infrastructure de test et autres testware pour une réutilisation ultérieure 4. Utiliser l'information recueillie pour améliorer la maturité du processus de test. ! Beaucoup de défauts sont des erreurs de codage → renforcer les tests statiques comme les revues de code ou avoir plus de tests unitaires

#planification plan de test, calendrier des tests, référentiel des risques et critères d'entré et de sortie

#Pilotage : apports d'avancement des tests, informations sur les risques

#Analyse conditions de test (priorisées), rapports de défauts

#Conception Cas de test (priorisés) chartes de test, **éléments de couverture** exigences en matière de données de test et exigences en matière d'environnement de

#Implémentation procédures de test, scripts de test automatisés, suites de tests, données de test, calendrier d'exécution des tests, et éléments de l'environnement de tes

#Exécution logs de test et rapports de défaut

#clôture rapport de clôture des tests, mesures à prendre pour améliorer les projet

Rôles dans le test

Le rôle de test manager axé sur les activités de planification des tests, de pilotage et de contrôle des tests et de clôture des tests (processus de test)

Le rôle de testeur est axé sur les activités d'analyse de test, de conception des tests, d'implémentation des tests et d'exécution des tests (aspect technique)

Compétences essentielles et bonnes pratiques en matière de test

Compétences génériques requise pour le test

La curiosité, esprit critique, souci du détail, esprit d'équipe, une bonne communication, connaissances techniques, connaissance du domaine, du pessimiste professionnel

Approche équipe intégrée

Dans l'approche intégrée, tout membre de l'équipe disposant des connaissances et des compétences nécessaires peut effectuer n'importe quelle tâche, et chacun est responsable de la qualité.

Les membres de l'équipe partagent le même espace de travail (physique ou virtuel)

Indépendance du test

Une indépendance est nécessaire pour bien tester.

Les produits d'activités peuvent être testés par :

- leur auteur (pas d'indépendance)
- les pairs de l'auteur appartenant à la même équipe (un peu d'indépendance)
- des testeurs extérieurs à l'équipe de l'auteur mais au sein de l'organisation (indépendance élevée)
- des personnes d'une organisation ou société différente (indépendance très élevée)

2. Tester pendant le cycle de vie du logiciel

Modèles de développement logiciels

Le test doit être adapté au cycle de vie du développement logiciel

Quel que soit le modèle de cycle de développement logiciel choisi, les activités de test devraient commencer dès les premières étapes du cycle de vie, en respectant le principe de « Tester tôt »

Développement séquentiel, les testeurs participent à la revue des exigences, à l'analyse et à la conception des tests. Le code exécutable est généralement créé dans les phases ultérieures, de sorte que les tests dynamiques ne peuvent généralement pas être effectués au début du cycle de vie du développement logiciel (en cascade, en v)

Développement incrémental et itératif : on suppose que chaque itération fournit un prototype fonctionnant ou un incrément de produit. Cela implique qu'à chaque itération, des tests statiques et dynamiques peuvent être effectués à tous les niveaux de test. La livraison fréquente d'incréments nécessite un feedback rapide et des tests de régression poussés

! **Modèle itératif** se concentre sur des cycles de DEV répétitifs pour une amélioration continue.

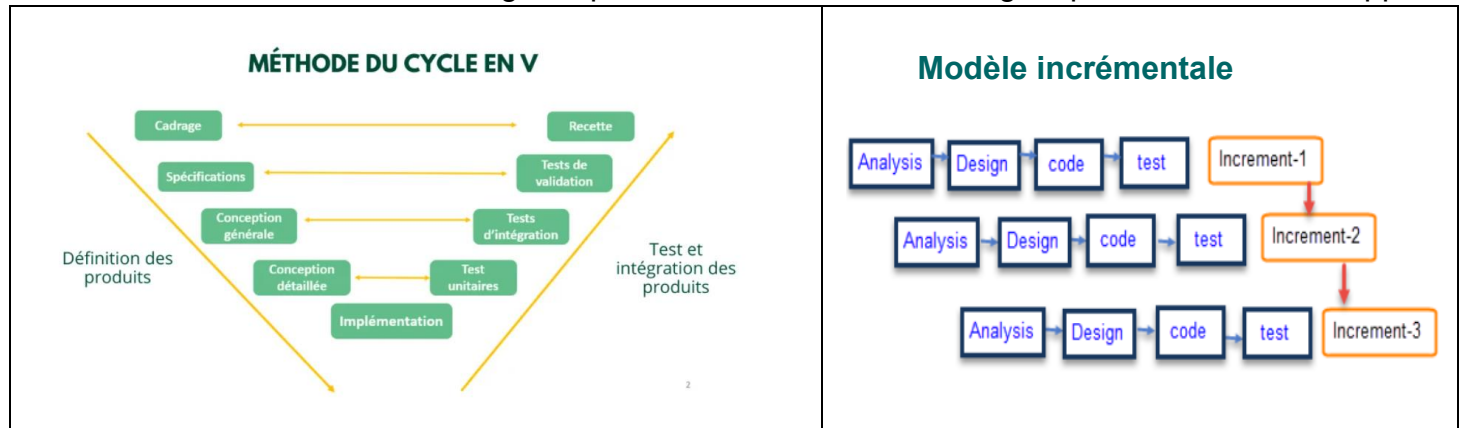
! **Modèle incrémental** se concentre sur la livraison progressive de fonctionnalités fonctionnelles en parties plus petites

Développement logiciel agile les projets agiles privilégient une documentation légère des produits d'activités et une automatisation poussée des tests pour faciliter les tests de régression. La plupart des

tests manuels tendent à être effectués à l'aide de techniques de test basées sur l'expérience qui ne nécessitent pas d'analyse et de conception des tests préalables approfondies.

Modèle en cascade : Toute phase du processus de développement devrait commencer lorsque la phase précédente est terminée.

Modèle en V : le modèle en V intègre le processus de test tout au long du processus de développement



Modèle itératif : Chaque itération délivre un logiciel opérationnel qui est un sous-ensemble croissant de l'ensemble global des caractéristiques jusqu'à ce que le logiciel final soit livré ou que le développement soit arrêté (**spirale, le prototypage**)

Cycle de vie du développement logiciel et bonnes pratiques de test

Pour chaque activité de développement logiciel, il existe une activité de test

Les testeurs sont impliqués dans la revue des produits d'activités dès que les versions préliminaires de cette documentation sont disponibles, de sorte que ces tests et cette détection de défauts plus **précoces** puissent soutenir la **stratégie shift left**

Le test en tant que moteur du développement de logiciels

TDD, **ATDD** et **BDD** sont des approches de développement similaires, dans lesquelles les tests sont définis comme un moyen d'orienter le développement.

Développement piloté par les tests (TDD):

Les tests sont écrits en premier, puis le code est écrit pour satisfaire les tests, et enfin les tests et le code sont refactorisés.

+Exp : les tests unitaires sont écrits en premier lieu (JUnit)

Développement piloté par les tests d'acceptation (ATDD):

Approche collaborative : Les tests sont écrits avant que la partie de l'application ne soit développée (avant l'implémentation des US) pour satisfaire aux tests. **Dériver les tests à partir des critères d'acceptation**

+Exp : Framework Cucumber

Développement piloté par le comportement (BDD):

Exprime le comportement souhaité d'une application avec des cas de test écrits dans une forme simple de langage naturel en utilisant généralement le format Given/When/Then (Etant donné/Lorsque/Alors)

+Exp : Framework Cucumber

DevOps et tests

DevOps est une approche organisationnelle visant à créer une synergie en amenant le développement (y compris le test) et l'exploitation à travailler ensemble pour atteindre un ensemble d'objectifs communs.

DevOps favorise l'autonomie des équipes, le retour d'information rapide, les chaînes d'outils intégrées et les pratiques techniques telles que l'intégration continue (**CI**) et la livraison continue (**CD**)

Cela permet aux équipes de construire, de tester et de livrer plus rapidement un code de haute qualité par le biais d'un pipeline de livraison DevOps

Approche shift left

Le principe du shift left suggère normalement que **les tests doivent être effectués plus tôt** (par exemple, sans attendre que le code soit implémenté ou que les composants soient intégrés), mais il ne signifie pas que les tests doivent être négligés dans les phases ultérieures du cycle de vie du développement logiciel.

Rétrospectives et amélioration de processus

Les rétrospectives (également appelées "réunions post-projet" et rétrospectives de projet) sont souvent organisées à la fin d'un projet ou d'une itération, lors d'une étape de livraison, ou peuvent être organisées en cas de besoin.

Répondre au 3 question :

Qu'est-ce qui a été un succès et qui devrait être conservé?

Qu'est-ce qui n'a pas fonctionné et qui pourrait être amélioré ?

Comment intégrer les améliorations et conserver les succès à l'avenir ?

Niveaux de tests

! Les technique de test se fait à n'importe quel niveau de test

1-Test de composant (unitaire) -développeur

#Se concentrent sur le test des composants isolés.

Nécessitent un support spécifique (des harnais * de tests ou des frameworks de tests unitaires).

Effectués par les **développeurs** dans leur environnement de test

Objets de test : Composants, Code et structures de données, Classes, Modules de bases de données

2-Tests d'intégration composant (unitaire) -développeur

Se concentrent sur le test des interfaces et des interactions entre les composants

Objets de test : Implémentation de bases de données des sous-systèmes, Infrastructure, Interfaces, APIs, Micro-services

3- Tests systèmes -Testeur

Se concentrent sur le comportement global et les capacités d'un système ou d'un produit entier, tester le comportement du système (de bout en bout Fonctionnels et Non Fonctionnels)

Objets de test :

Applications, Systèmes Matériel/Logiciel, Systèmes d'exploitation, Configuration du système et données de configuration.

4-Tests d'intégration système -Testeur

Se concentrent sur le tester les interfaces entre le système sous test et d'autres systèmes et services externes dans un environnement similaires à l'environnement opérationnel

5- Tests d'acceptation -PO

Se concentrent la validation et la démonstration de l'aptitude au déploiement, la réponse aux besoins métier de l'utilisateur, effectués par les utilisateurs prévus

Les formes communes de tests d'acceptation:

1- Tests d'acceptation **utilisateur**

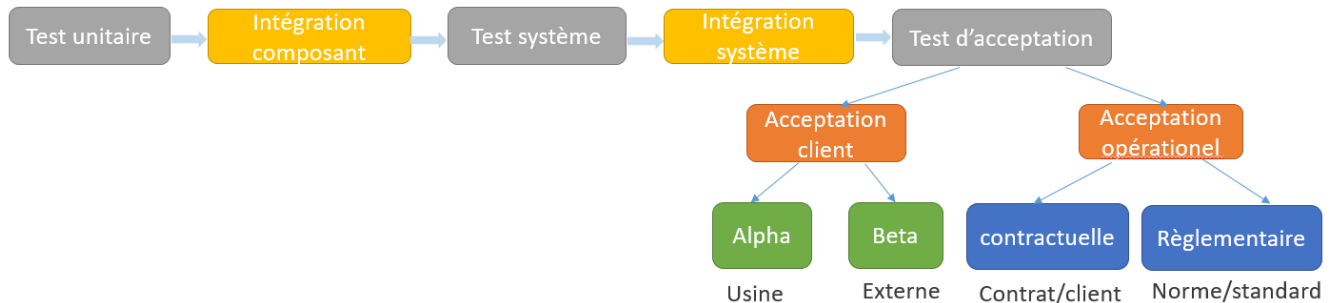
2- Tests d'acceptation **opérationnelle** (sur terrain sur l'environnement de production)

3- Tests d'acceptation **contractuelle** (axée sur les engagements du fournisseur de développement vis-à-vis de son client) et **réglementaire** (axée sur l'application de règlements internes et externes standards)

4- Tests alpha et bêta

ALPHA TEST (acceptation usine ou interne)
Sur le site du développement
 En dehors de l'organisation du développement
 Réalisé par les utilisateurs potentiels ou équipes indépendantes

BETA TEST (acceptation sur site ou externe)
 Sur un **site externe au développement** (sur le terrain)
 En dehors de l'organisation du développement
 Réalisé par les utilisateurs potentiels ou équipes indépendantes



Type de test :

Fonctionnel

Évalue les fonctions qu'un composant ou un système es fonctions sont **"ce que" l'objet de test doit faire**

Non fonctionnel

C'est le test de **"comment le système se comporte"** dont l'objectif est de vérifier les caractéristiques/qualité non fonctionnelles du logiciel

Les tests non fonctionnels commencent tôt dans le cycle de vie du DEV, découverte tardive de défauts non fonctionnels peut constituer une menace sérieuse pour la réussite d'un projet.

(Efficacité de la performance, Compatibilité, Utilisabilité, Fiabilité, Sécurité, Maintenabilité, Portabilité)

Boîte noire

Basé sur les spécifications et dérive les tests de la documentation externe à l'objet de test. L'objectif principal du test boîte noire est de **vérifier le comportement du système par rapport à ses spécifications**.

Le test boîte blanche

Basé sur la structure et dérive les tests de l'implémentation ou de **la structure interne du système** (par exemple, le code, l'architecture, les **flux métier** et les **flux de données**).

! Les quatre types de tests peuvent être appliqués à tous les niveaux de test

Les tests liés aux changements :

Les tests devraient inclure des tests de confirmation et des tests de régression après les modifications apportées à un composant ou à un système, soit pour l'améliorer par l'ajout de nouvelle caractéristique soit pour le corriger par l'élimination d'un défaut

Confirmation: confirmer que les défauts originaux ont été corrigés,

Régression: pour rechercher tout nouveau défaut lié à des modifications du logiciel ou de son envi

Test de maintenance

La maintenance est l'ensemble d'activités effectuées sur le logiciel après le déploiement.

Il existe **3 déclencheurs** des tests de maintenance :

Modification : des améliorations planifiées, des changements correctifs et d'urgence (Hot-fix), des changements de l'environnement opérationnel, des correctifs pour les défauts et les vulnérabilités.

Migration : d'une plate-forme à une autre, migration des données.

Déclassement : retrait/mitigation lorsqu'une application arrive en fin de vie.

Test de maintenance se fait via :

- Des Batch : correction facile sur le champ
- Retour en arrière avec une version précédente stable (elle ne doit pas prendre beaucoup de temps)

! Les tests de maintenance ne peuvent pas être automatisé (très coûteux)

3. Techniques statiques

Tests statiques : reposent sur l'examen manuel (revues) ou l'analyse (analyse statique) du code source (non exécuté) ou de la documentation du projet

Contrairement aux tests dynamiques, les techniques statiques **trouvent les causes des défauts** plutôt que les défaillances elles-mêmes

Produits d'activités peuvent être examinés à l'aide de tests statiques :

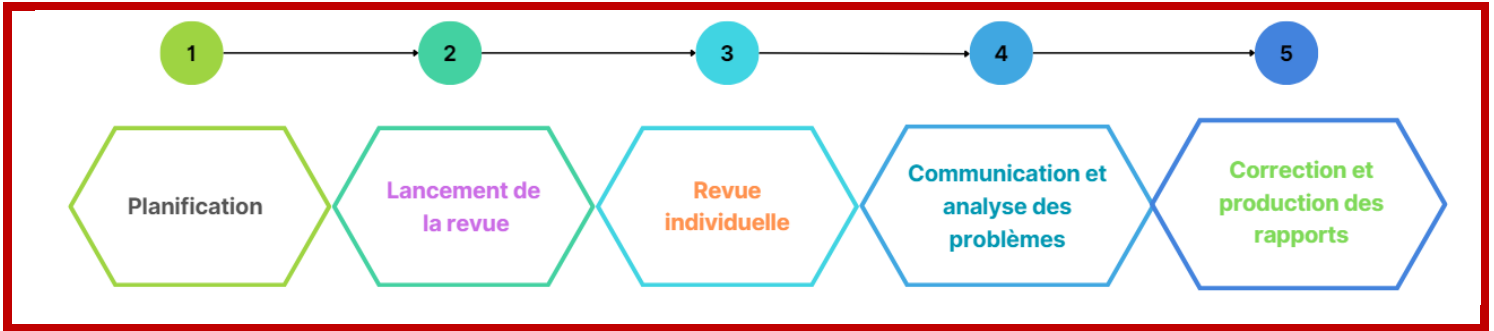
- Les spécifications (les exigences métier, les exigences fonctionnelles, les exigences de sécurité)
- Le testware (les plans de test, les cas de test, les procédures de test, les scripts de test automatisés)
- Les épics, User Stories, et critères d'acceptation
- Les spécifications d'architecture et de conception
- Le code
- Les guides utilisateur
- Les pages Web
- Les contrats, les plans de projet, les calendriers et les budgets
- Les modèles, tels que les diagrammes d'activité

Les défauts typiques : déviations par rapport aux standards, défauts d'exigences, défauts de conception, maintenabilité insuffisante et spécifications incorrectes d'interfaces.

Les revues informelles se caractérisent par le fait qu'elles ne suivent pas un processus défini et n'ont pas de résultats formels documentés.

Les revues formelles sont caractérisées par la participation de l'équipe, la documentation des résultats de la revue et la documentation des procédures pour la conduite de la revue

Processus d'une revue formelle



1- Planification:

Définir le périmètre de la revue: l'objectif de la revue, le produit d'activités à examiner, les critères d'entrée/ sortie, l'effort et les délais de la revue

2- Lancement de la revue

Objectif est de s'assurer que toutes les personnes impliquées sont prêtes à commencer la revue. S'assurer que chaque participant a accès au produit d'activités examiné, qu'il comprend son rôle et ses responsabilités et qu'il reçoit tout ce qui est nécessaire à l'exécution de la revue

3- Revue individuelle (c.-à-d. préparation individuelle)

Noter les défauts potentiels, les recommandations et les questions

4- Communication et analyse des problèmes

- Communiquer les défauts potentiels identifiés
- Analyser les défauts potentiels, leur affecter un responsable et un statut
- Évaluer et documenter les caractéristiques de qualité
- Évaluer les résultats de la revue en fonction des critères de sortie pour prendre une décision de revue

5- Correction et production de rapports

Pour chaque défaut, **un rapport de défaut doit être créé** afin que les actions correctives puissent faire l'objet d'un suivi.

- Lorsque les critères de sortie sont atteints, le produit d'activités peut être accepté. Les résultats de la revue font l'objet d'un rapport

Les acteurs d'une revue :

Auteur : crée le produit d'activités revu, corrige les défauts du produit d'activités revu (si nécessaire)

Manager : **décide** de **ce qui doit être revu** et fournit les ressources, telles que le personnel et le temps nécessaires à la réalisation de la revue.

Modérateur (facilitateur) : veille au déroulement efficace des réunions de revue, y compris la médiation, la gestion du temps et un environnement de revue sûr dans lequel chacun peut s'exprimer librement.

Réviseurs : effectue les revues. Un réviseur peut être une personne travaillant sur le projet, un expert en la matière avec une culture technique ou métier spécifique (aussi appelés vérificateurs ou inspecteurs).

Scribe : documente tous les aspects, problèmes et points ouverts identifiés pendant la réunion (décisions et les nouvelles anomalies trouvées)

Responsable de la revue prend la responsabilité générale de la revue, décide qui sera impliqué et organise quand et où elle aura lieu

Types de revue

- **Informelle** (par un collègue, revue de pair) ne suit pas un processus défini et n'exigent pas de résultat formel documenté.
- **Relecture technique** (revue de code) (walkthrough) : (souvent formel entre les développeurs /testeurs)
 - Menée par l'auteur
 - But : **évaluation de la qualité** et le renforcement de la confiance dans le produit d'activités, a formation des réviseurs, l'obtention d'un consensus, la génération de nouvelles idées, la motivation et la capacité des auteurs à s'améliorer et la détection d'anomalies
 - Les réviseurs peuvent effectuer une revue individuelle avant la relecture technique, mais ce n'est pas obligatoire.
- **Revue technique formel**
 - **Réalisée par des réviseurs techniquement qualifiés**
 - Dirigée par un modérateur
 - But : parvenir à un consensus (prévoir) et de prendre des décisions concernant un problème technique, détecter des anomalies, d'évaluer la qualité et de construire la confiance dans le produit d'activités motiver les auteurs et de leur permettre de s'améliorer
- **Inspection** le plus **formel**, elles suivent le processus générique complet,
 - But : trouver le maximum d'anomalies
 - Des métriques sont collectées et utilisées pour améliorer le cycle de vie de développement du logiciel, y compris le processus d'inspection
 - **lors des inspections l'auteur ne peut pas jouer le rôle de réviseur ou de scribe**

Facteurs de réussite des revues

- Définir des objectifs clairs et des critères de sortie mesurables
- Choisir le type de revue approprié pour atteindre les objectifs fixés
- Réaliser des revues en petits groupes
- Fournir un feedback des revues aux parties prenantes et aux auteurs
- Accorder aux participants suffisamment de temps pour se préparer à la revue.
- Bénéficier d'un soutien, de la part du management,
- Intégrer les revues dans la culture de l'organisation
- Fournir une formation adéquate à tous les participants et faciliter la réunion

4. Techniques de conception de Tests

Catégories de techniques de conception de test

Technique boîte noir	Technique boîte blanche	Basé sur l'expérience
- Partition d'équivalence - Analyse valeurs limites (2 et 3 valeurs) - Tables décisions - Transition d'état	- Couverture d'instruction - Couverture des branches - Couverture des conditions - Couverture des décisions	- Estimation d'erreur - Test exploratoire - Test basé sur les checklist

(1) Technique Boîte noir

(Sans références internes) Basée sur le comportement externe

Techniques associées

Partitions d'équivalences

Tous les éléments d'une partition donnée doivent être traités de la même manière par l'objet de test

Les partitions peuvent être continues ou discrètes, ordonnées ou non, finies ou infinies.

Les partitions ne doivent pas se chevaucher et doivent être des ensembles non vides

Couverture de 100 % → exercer toutes les partitions identifiées (y compris les partition non valides)

Analyse valeurs limites

Ne peut être utilisée que pour des partitions ordonnées.

Dans la technique à **2 valeurs**, pour chaque valeur limite, il y a deux éléments de couverture : la valeur limite en question et sa voisine la plus proche appartenant à la partition adjacente (**valeurs adjacentes à l'extérieur**)

Couverture = nb valeurs limite exercées / nb valeurs limite identifiés

Couverture de 100 % → exercer toutes les valeurs limites identifiées

Dans la technique à **3 valeurs** on prend la valeur limite en question et ses deux voisines (**valeurs adjacentes à l'extérieur et l'intérieur de la partition.**)

Tables de décisions

Tester les interactions entre des combinaisons de conditions dans le but d'assurer que toutes les combinaisons de conditions sont testées

Transition d'états

Permet de tester la capacité d'un système à entrer dans des états définis et à en sortir par des transitions valides et invalides.

Couverture de tout l'état

Couverture = nb d'états visités / nb total d'états

Couverture de toutes les transitions,

Couverture = nb de transitions valides et non valides exercées ou tentées d'être exercées / nb total de transitions valides et non valides

Couverture des transitions valides (également appelée couverture 0-switch)

Couverture = nb de transitions valides exercées / nb total de transitions valides

La couverture de tous les états est plus faible que la couverture des transitions valides

(2) Technique Boîte blanche

Utilisation de la structure du composant

Utilisation des chemins internes du composant

Techniques associées

Couverture instruction

Couverture branches bloc de base qui peut être sélectionné pour exécution, basé sur une construction dans laquelle un chemin, parmi deux ou plus, est disponible: case, jump, go to, if-then-else

Couverture décisions: un nœud avec deux ou plus liens vers des branches séparée

Couverture conditions: expression logique qui peut être évaluée à Vrai ou Faux.

$V(G)$ donne le nombre de chemins indépendants dans un algorithme

$$V(G) = e - n + NbS + NbE$$

e = nombre d'arcs, n = nombre de nœuds, NbE / NbS = nombre d'entrées / sorties.

Couverture d'instructions et branches

La couverture des branches inclut la couverture des instructions (càd la satisfaction couverture des branches garantie la satisfaction de couverture des instructions)

Décisions & conditions

Couverture des décisions (prédicat* composé) il faut couvrir la décision et la contre décision

(3)Techniques basées sur l'expérience

Tirent parti de l'expérience des développeurs, des testeurs et des utilisateurs

Estimation d'erreurs : basée sur l'expérience personnelle de chaque testeur **anticiper les erreurs**

Les tests exploratoires : une approche basée sur des sessions pour structurer l'activité où conception, exécution, écriture des résultats de tests sont effectués en même temps que l'apprentissage de l'application, sur base de périodes de temps limitées (blocs) et d'objectifs définis par avance (charte)

Utile si manque de spécifications, contraintes de temps fortes.

Tests basés sur des checklist : les testeurs conçoivent, implémentent et exécutent des tests pour couvrir les conditions de test figurant dans une checklist

! Attaque par défaut : Ceci est une liste d'échecs possibles. Les attaques de fautes sont une approche méthodique de la mise en œuvre de la détection des erreurs et nécessitent que le testeur crée ou acquière une liste d'erreurs,

Approches de test basées sur la collaboration

Rédaction collaborative de User Stories

Le format le plus courant d'une US est le suivant : "En tant que [rôle], je veux que [objectif à atteindre], afin de pouvoir [valeur métier résultante pour le rôle]", suivi des critères d'acceptation.

Les US présentent trois aspects essentiels "3 C" :

- **C**arte- le support décrivant une US (exp : une carte d'index, une entrée dans un tableau électronique)
- **C**onversation - explique comment le logiciel sera utilisé (peut être documentée ou verbale)
- **C**onfirmation - les critères d'acceptation

User Story doit être : Indépendantes, Négociables, apportant de la Valeur, Estimables, Petites et Testables.

Critères d'acceptation

User Story. Les deux formats les plus courants pour rédiger des critères d'acceptation des US:

Orienté-scénario (exp, le format Given/When/Then (Etant donné/Lorsque/Alors) utilisé par le BDD)

Orienté vers les règles (exp, liste de vérification à puces, ou tableau de correspondance entrée-sortie)

Développement piloté par les tests d'acceptation (ATDD):

Les tests sont écrits avant d'implémenter l'User Story

- En général, les premiers cas de test sont positifs, confirmant le comportement correct sans exceptions ou conditions d'erreur
- Une fois les cas de test positifs réalisés, l'équipe doit effectuer des tests négatifs.
- Enfin, l'équipe doit également couvrir les caractéristiques non fonctionnelles de la qualité

(1) Cas passant → (2) cas non passant → (3) caractéristiques non fonctionnelles

5. Gestion des activités de test

1-Planification des tests

Objet de plan test

- Documente les moyens et le calendrier pour atteindre les objectifs de test.
- Aide à garantir que les activités de test réalisées répondront aux critères établis.
- Sert de moyen de communication avec les membres de l'équipe et les autres parties prenantes.
- Démontre que le test sera conforme à la politique de test et à la stratégie de test existantes (ou explique pourquoi le test s'en écarte)

Contenu d'un plan de test

Le contenu typique d'un plan de test comprend:

- Le contexte du test (périmètre, objectifs de test, contraintes, base de test)
- Les hypothèses et contraintes du projet de test
- Les parties prenantes (rôles, responsabilités, pertinence pour les tests, besoins en matière d'embauche et de formation)
- La communication formes et fréquence de communication, modèles de documentation)
- Le référentiel des risques (risques produits, risques projet)
- L'approche de test (niveaux de test, types de test, techniques de test...)
- Le budget et le calendrier

Contribution du testeur à la planification des itérations et des releases

Dans les cycles de vie de développement du logiciel itératifs, il existe généralement deux types de planification : la planification de la release et la planification de l'itération.

La planification de la release

Prévoit la livraison d'un produit

Les testeurs impliqués dans la planification de la release

- Définit et redéfinit le Product backlog,
- Participent à la rédaction des US testables et des critères d'acceptation
- Participent à l'analyse des risques projet et des risques qualité
- Décomposent les User Stories important en US plus petit
- Estiment l'effort de test associé aux User Stories
- Déterminent l'approche de test
- Planifient les tests pour la release.

La planification de l'itération

Se projette à la fin d'une seule itération et se préoccupe du backlog de l'itération.

Les testeurs impliqués dans la planification de l'itération:

- Participent à l'analyse détaillée des risques des User Stories
- Déterminent la testabilité des User Stories
- Décomposer les User Stories en tâches (en particulier les tâches de test)
- Estiment l'effort de test pour toutes les tâches de test et identifient et affinent les aspects fonctionnels et non fonctionnels de l'objet test.

Critères d'entrée et critères de sortie

Les critères d'entrée (Agile définition du prêt **DOR** definition of ready) définissent les préconditions pour entreprendre une activité de test donnée.

Exp :

- La disponibilité des ressources (le personnel, les outils, les environnements, les données de test, le budget, le temps)

- La disponibilité du matériel de test (la base de test, les exigences testables, les US, les cas de test), et le niveau de qualité initial d'un objet de test (tous les smoke tests ont été passés avec succès).

Les critères de sortie (Agile définition du terminé **DOD** definition of done) définissent les conditions métrique à remplir pour pouvoir déclarer un niveau de test ou un ensemble de tests terminés.

Exp :

- Les mesures de l'exhaustivité (le niveau de couverture atteint, le nombre de défauts non résolus, la densité de défauts, le nombre de cas de test en échec)

- Les critères de clôture (les tests planifiés ont été exécutés, les tests statiques ont été effectués, tous les défauts trouvés ont été signalés, tous les tests de régression ont été automatisés).

- L'épuisement du temps ou du budget

- Même si les autres critères de sortie ne sont pas satisfaits, il peut être acceptable de mettre fin au test dans de telles circonstances, si les parties prenantes ont revu et accepté le risque d'une mise en production sans tests supplémentaires.

Techniques d'estimation

Estimation basée sur des ratios:

Basée sur des métriques, des chiffres sont collectés à partir de projets **antérieurs** au sein de l'organisation, ce qui permet de déduire des ratios "standards" pour des projets similaires

Exp :

Ratio **3** : **2** (**DEV** / **TEST**) (! dans le QCM il peut écrire le ratio sous forme de **TEST/DEV**)

Activité de DEV : 600 h

Activité de test = $(600 \times 2) / 3$ (400 h)

Extrapolation:

Basée sur les métriques, des mesures sont effectuées le plus tôt possible dans le **projet en cours**

afin de recueillir les données. Lorsque l'on dispose de **suffisamment d'observations**, l'effort requis pour le reste du travail peut être estimé en extrapolant ces données.

Utilisé dans le modèle itérative

Delphi large bande

Technique itérative

Basée sur l'expertise, des experts font des estimations basées sur l'expérience. Chaque expert, de manière isolée, évalue l'effort. Les résultats sont collectés et s'il y a des écarts par rapport aux limites convenues, **les experts discutent de leurs estimations actuelles**.

Estimation en trois points

Basée sur l'expertise, trois estimations sont faites par des experts :

L'estimation la plus **optimiste (a)**

L'estimation la plus **probable (m)**

L'estimation la plus **pessimiste (b)**.

L'estimation finale (E)

$$E = (a + 4 \times m + b) / 6$$

L'avantage de cette technique est qu'elle permet aux experts de calculer l'erreur de mesure :

$$SD = (b - a) / 6$$

Priorisation des cas de test

Priorisation basée sur les risques, l'ordre d'exécution des tests est piloté par les résultats de l'analyse des risques. **Les cas de test couvrant les risques les plus importants sont exécutés en premier**

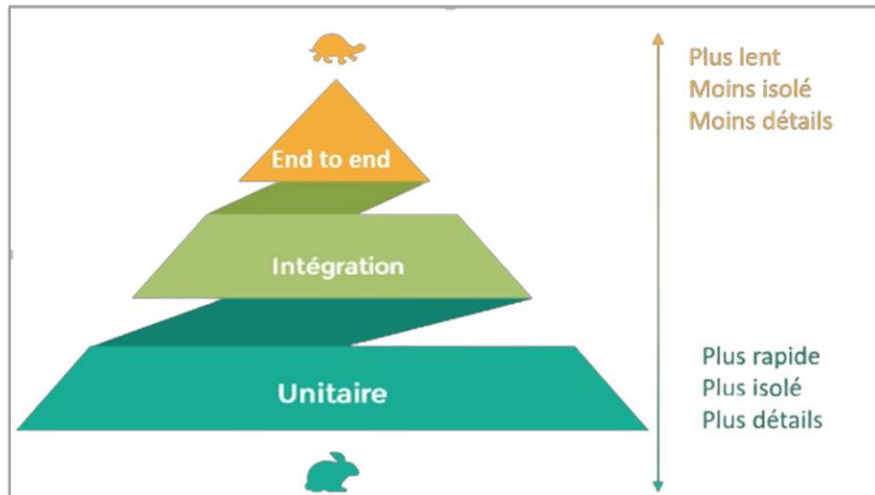
Priorisation basée sur la couverture, l'ordre d'exécution des tests est basé sur la couverture (par exemple, la couverture des instructions). **Les cas de test ayant la couverture la plus élevée sont exécutés en premier.**

Priorisation basée sur les exigences, l'ordre d'exécution des tests est basé sur les priorités des exigences (priorisé par les parties prenantes). **Les cas de test liés aux exigences les plus importantes sont exécutés en premier.**

Cette pratique peut ne pas fonctionner si les cas de test ou les caractéristiques testées ont des dépendances (C1 de priorité moins élevée que C2 et C1 dépend de C2 → C1 exécuté en premier)

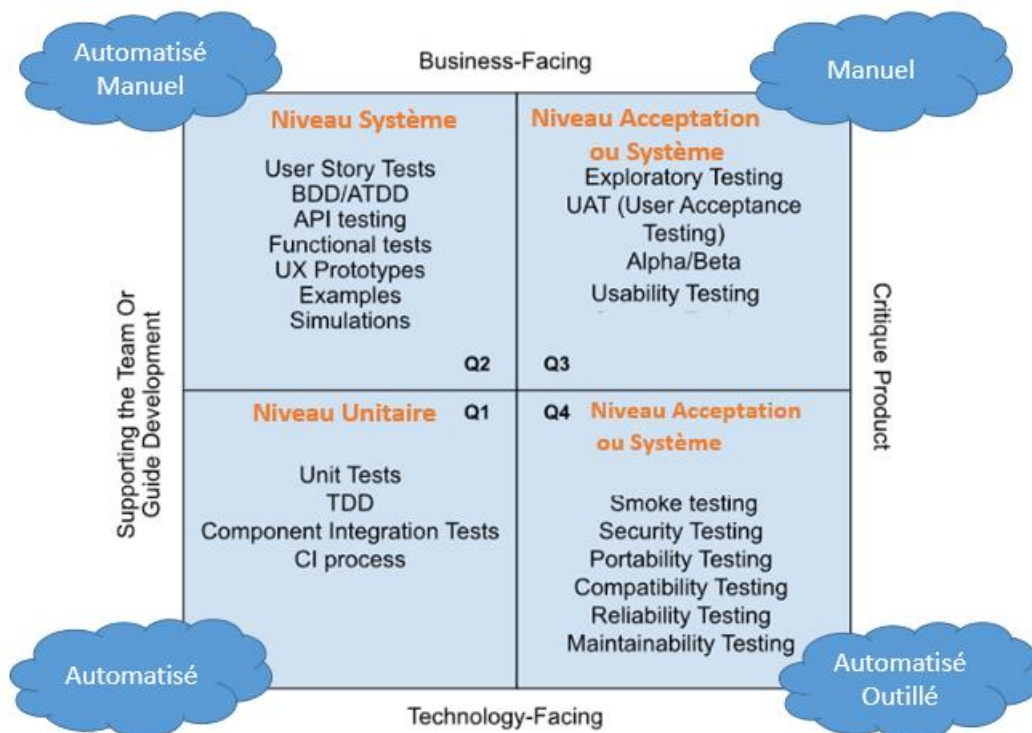
Pyramide des tests

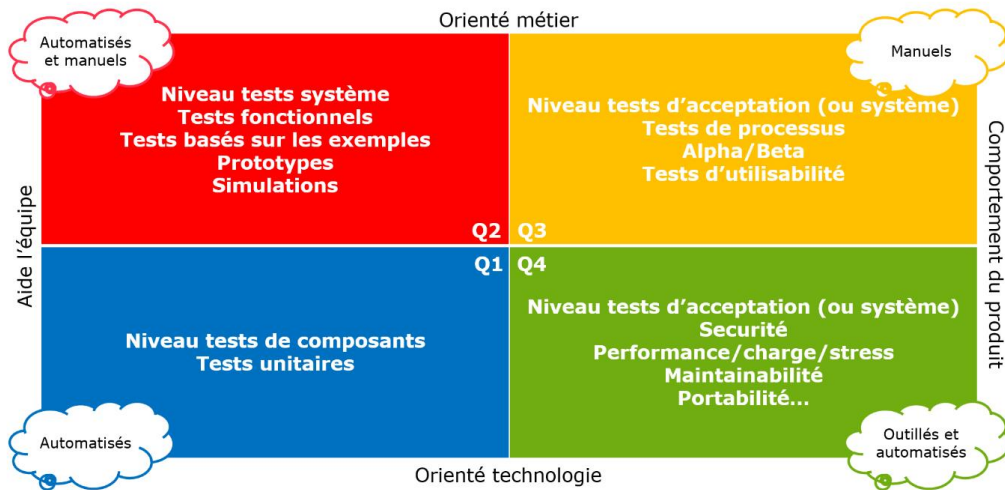
Un modèle qui montre que différents tests peuvent avoir un niveau de détail différent. Et qui aide l'équipe dans l'automatisation des tests et dans l'allocation de l'effort de test en montrant que différents objectifs sont soutenus par différents niveaux d'automatisation des tests. Les couches de la pyramide représentent des groupes de tests.



Quadrant de test

Permet d'aligner les niveaux de tests avec les types de tests appropriés, s'assurer que tous les types et niveaux de test appropriés sont inclus dans le cycle de vie du développement logiciel,





Quadrant Q1 (orienté vers la technologie, soutien à l'équipe). Ce quadrant contient les tests de composants et d'intégration de composants. Ces tests doivent être automatisés et inclus dans le processus d'intégration continue.

Quadrant Q2 (orienté vers le métier, soutien à l'équipe). Ce quadrant contient des tests fonctionnels, des exemples, des tests de User Story, des prototypes d'expérience utilisateur, des tests d'API et des simulations. Ces tests vérifient les critères d'acceptation et peuvent être manuels ou automatisés.

Quadrant Q3 (orienté vers le métier, critique du produit). Ce quadrant contient les tests exploratoires, les tests d'utilisabilité, les tests d'acceptation utilisateurs. Ces tests sont orientés vers l'utilisateur et souvent manuels.

Quadrant Q4 (orienté vers la technologie, critique du produit). Ce quadrant contient les smoke tests et les tests non fonctionnels (à l'exception des tests d'utilisabilité). Ces tests sont souvent automatisés.

2-Gestion des risques

La gestion des risques permet aux organisations d'augmenter la probabilité d'atteindre leurs objectifs, d'améliorer la qualité de leurs produits et d'accroître la confiance des parties prenantes.

Activités de gestion des risques sont les suivantes :

- **Analyse des risques** (comprend l'identification des risques et l'évaluation des risques)
- **Contrôle des risques** (comprend l'atténuation des risques et le pilotage des risques)

Test basé sur les risques : activités de test sont sélectionnées, priorisées et gérées sur la base de l'analyse des risques et du contrôle des risques

Définition du risque et attributs du risque

Un risque peut être caractérisé par deux facteurs :

- **La probabilité du risque** de se produire
- **L'impact du risque (préjudice)** les conséquences de cette occurrence.

Niveau de risque = occurrence (probabilité) * impact

Risques projet et risques produit

Les risques projet : liés à la gestion et au contrôle (gestion) du projet.

Les problèmes organisationnels : des retards dans la livraison des produits d'activités, des estimations inexactes, réduction des coûts

Les problèmes humains : compétences insuffisantes, conflits, problèmes de communication, manque de personnel

Les problèmes techniques : dépassement du périmètre, manque de soutien des outils

Les problèmes liés aux fournisseurs

Impact

Sur le calendrier, le budget ou le périmètre du projet, ce qui affecte la capacité du projet à atteindre ses objectifs.

Les risques produits : liés aux caractéristiques-qualité du produit.

- Fonctionnalités manquantes ou incorrectes

- Calculs erronés
- Erreurs d'exécution
- Architecture médiocre
- Algorithmes inefficaces
- Temps de réponse inadéquat
- Mauvaise expérience utilisateur
- Vulnérabilités de sécurité

Impact

- L'insatisfaction des utilisateurs.
- La perte de revenus, de confiance et de réputation.
- Des dommages causés à des tiers.
- Des coûts de maintenance élevés, une surcharge du service d'assistance.
- Des sanctions pénales.
- Dans les cas extrêmes, des dommages physiques, des blessures ou même des décès.

Analyse des risques produits

L'analyse des risques produit comprend l'identification des risques et l'évaluation des risques.

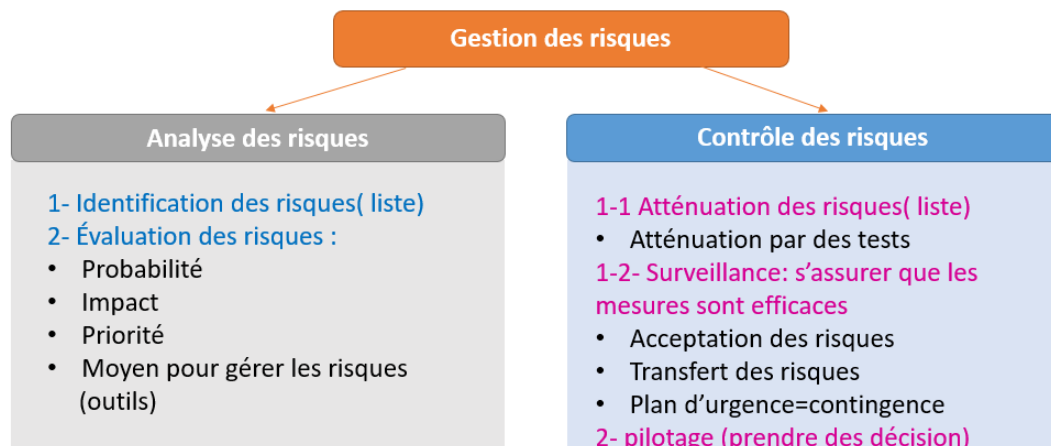
L'identification des risques consiste à **dresser une liste exhaustive des risques**.

Contrôle des risques produits

Le contrôle des risques produits comprend l'atténuation des risques et la surveillance des risques.

#Les mesures qui peuvent être prises pour réduire les risques par des tests

- Sélectionner les testeurs avec bon niveau d'expérience et de compétences, adaptés à un type de risque donné.
- Appliquer un niveau approprié d'indépendance du test.
- Effectuer des revues et réaliser des analyses statiques.
- Appliquer les techniques de test et les niveaux de couverture appropriés.
- Appliquer les types de tests appropriés en fonction des caractéristiques qualités concernées
- Effectuer des tests dynamiques, y compris des tests de régression



Pilotage des tests, contrôle des tests et clôture des tests

Le pilotage des tests concerne la collecte d'informations sur les tests qui sont utilisées pour évaluer l'avancement des tests et pour mesurer si les critères de sortie des tests ou les tâches de test associées aux critères de sortie sont satisfaits,

Le contrôle des tests utilise les informations issues du pilotage des tests pour fournir, sous la forme de **directives de contrôle**

Exemples de directives de contrôle:

- Redéfinir l'ordre de priorité des tests lorsqu'un risque identifié devient un problème.
- Réévaluer si un élément de test répond aux critères d'entrée ou de sortie en raison d'un changement.
- Ajuster le calendrier des tests pour tenir compte d'un retard dans la livraison de l'environnement de test.
- Ajouter de nouvelles ressources en cas de besoin.

Métriques utilisées pour les tests

- **Métriques d'avancement du projet** (clôture des tâches, utilisation des ressources, effort de test).
- **Métriques d'avancement des tests** (avancement de l'implémentation des cas de test, avancement de la préparation de l'env de test, nombre de cas de test exécutés, réussis/échecs, durée d'exécution)
- **Métriques de qualité du produit** (disponibilité, temps de réponse, temps moyen jusqu'à la défaillance)
- **Métriques relatives aux défauts** (nombre et priorités des défauts trouvés/corrigés, densité des défauts, pourcentage de détection des défauts).
- **Métriques de risque** (de risque résiduel).
- **Métriques de couverture** (couverture des exigences, couverture du code).
- **Métriques de coût** (coût du test, coût organisationnelle la qualité).

Objet, contenu et destinataires des rapports de tests

Les rapports d'avancement contribuent au **contrôle continu** des tests et doivent fournir suffisamment d'informations pour permettre de **modifier le calendrier des tests, les ressources ou le plan de test**. Les rapports d'avancement comprennent les éléments suivants:

- Période de test.
- Progression des tests (par exemple, avance ou retard par rapport au calendrier), y compris tout écart notable.
- Obstacles aux tests et leurs solutions de contournement.
- Métriques de test.
- Risques nouveaux et modifiés au cours de la période de test.
- Tests planifiés pour la prochaine période.

Un rapport de clôture (Synthèse) des tests est préparé lors de la clôture des tests, lorsqu'un projet, un niveau de test ou un type de test est terminé et que, **dans l'idéal, ses critères de sortie ont été remplis**. Les rapports de clôture des tests les plus courants sont les suivants:

- Résumé du test.
- Évaluation du test et de la qualité du produit sur la base du plan de test initial (c'est-à-dire les objectifs du test et les critères de sortie)
- Écarts par rapport au plan de test (différences par rapport au calendrier, à la durée et à l'effort prévus).
- Obstacles au test et solutions de contournement.
- Métriques de tests basés sur les rapports d'avancement des tests.
- Risques non atténués, défauts non corrigés.
- Leçons apprises pertinentes pour le test.

Communication de l'état d'avancement des tests

- Communication verbale avec les membres de l'équipe et les autres parties prenantes.
- Tableaux de bord (tableaux de bord intégration continue/développement continu, tableaux des tâches et burn-down charts*)
- Canaux de communication électronique (mail, chat).
- Documentation en ligne.
- Rapports de tests formels

Gestion de configuration

Si l'élément de configuration est approuvé pour les tests, il devient une base de référence et **ne peut être modifié** que dans le cadre d'un processus formel de contrôle des modifications.

L'intégration continue, la livraison continue, le déploiement continu et les tests associés sont généralement implémentés dans le cadre d'un pipeline DevOps automatisé, dans lequel la gestion de configuration automatisée est normalement incluse.

Gestion des défauts

Les défauts trouvés pendant les tests devraient être enregistrés. Tout défaut identifié doit faire l'objet d'une analyse et faire l'objet d'un suivi depuis sa découverte et sa classification jusqu'à sa résolution. Les testeurs devraient faire en sorte de réduire au minimum le nombre de faux positifs signalés comme étant des défauts.

Le rapport de défauts

Les défauts peuvent être rapportés pendant le codage, l'analyse statique, les revues, les tests dynamiques ou l'utilisation d'un produit logiciel

Un rapport de défaut rempli pendant les tests dynamiques comprend généralement les éléments suivants:

- **Un identifiant**
- **Un titre** et un bref résumé
- **La date du rapport de défaut**, **l'organisation émettrice** et **l'auteur**, y compris son rôle
- **L'identification de l'élément de test et de l'environnement de test**
- **Contexte du défaut** (**cas** de test en cours d'exécution, activité de test en cours, phases du cycle de développement au cours desquelles le défaut a été observé)
- **Une description** du défaut pour permettre la reproduction et la résolution, y compris des logs, des captures de base de données, des captures d'écran ou des enregistrements (si trouvés pendant l'exécution des tests)
- **Les résultats attendus et obtenus**
- **La portée ou le degré d'impact (sévérité)** du défaut pour les parties prenantes concernées
- **Urgence/priorité** de la correction
- **Statut du défaut** (par exemple, ouvert, différé, dupliqué, en attente de correction, en attente de test de confirmation, ré-ouvert, fermé, rejeté).

6. Outils de test

- **Outils de Gestion** - augmentent l'efficacité du processus de test en facilitant la gestion du cycle de vie du développement logiciel, des exigences, des tests, des défauts et de la configuration.
- **Outils de test statique** - aident le testeur à effectuer des revues et des analyses statiques.
- **Outils de conception et d'implémentation des tests** - facilitent la génération des cas de test, des données de test et des procédures de test.
- **Outils d'exécution des tests et de couverture** - facilitent l'exécution automatisée des tests et la mesure de la couverture.
- **Outils de tests non fonctionnels** - permettent au testeur d'effectuer des tests non fonctionnels difficiles ou impossibles à réaliser manuellement.
- **Outils DevOps** - soutiennent le pipeline de livraison DevOps, le suivi du workflow, le(s) processus de build automatisé(s), l'intégration continue/le développement continu.
- **Outils de collaboration** - facilitent la communication
- **Outils prenant en charge l'évolutivité et la standardisation du déploiement** (par exemple, machines virtuelles, outils de conteneurisation)

Avantages de l'automatisation des tests

Prévention des erreurs humaines simples

Gain de temps grâce à la réduction du travail manuel répétitif

Évaluation plus objective (par exemple, de la couverture)

Accès plus facile à l'information sur les tests pour soutenir la gestion des tests et le reporting des tests

Réduction des délais d'exécution des tests pour une détection plus précoce des défauts

Risques de l'automatisation des tests

- Estimations inexactes du temps
- Utilisation d'un outil de test alors que les tests manuels sont plus appropriés.
- Trop grande dépendance à l'égard d'un outil,
- Dépendance à l'égard du fournisseur de l'outil
- Utilisation d'un logiciel libre qui peut être abandonnée,
- Incompatibilité de l'outil d'automatisation avec la plateforme de développement.
- Choix d'un outil inadapté qui n'a pas respecté les exigences réglementaires et/ou les standards de sécurité.

Termes et notes:

Heuristique de test : contient tous les outils ou l'environnement nécessaire pour faire les tests

Harnais de test : les ressources nécessaires pour exécuter un test des pilotes (**drivers** : Remplace ou simule le composant appelant) et des bouchons (**stubs** : remplace ou simule le composant appelé)

Indicateurs de test : sont des mesures clés utilisées pour évaluer et suivre la qualité des tests effectués Certains indicateurs courants incluent le First Pass Yield, qui évalue la qualité des livraisons fréquentes, et d'autres **KPIs (Key Performance Indicators)** qui mesurent le succès des tests automatisés. Informations sur quand-est-ce-que les tests sont terminée et quand ils sont stoppés

Compagne de test : c'est un ensemble des instances d'un ou plusieurs plans de test

Patrimoine de test = référentiel de test: une sorte d'archive contenant (plan de test, documentation, test non fonctionnels...)

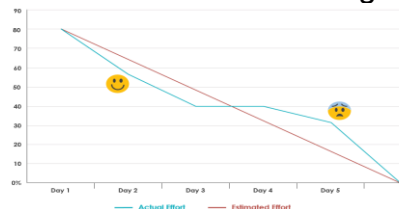
Oracle de test : Une source utilisée pour déterminer les résultats attendus à comparer avec les résultats obtenus de l'application en cours de tests

Charte de test : Documentation des activités de test dans le cadre de tests exploratoires basés sur des sessions.

Périmètre de test : qu'est-ce qu'on doit et qu'on ne doit pas tester

Condition de test : un article ou événement d'un composant ou système qui pourrait être vérifié par un ou plusieurs cas de tests; p.ex. Une fonction, une transaction, un attribut qualité ou un élément de structure.

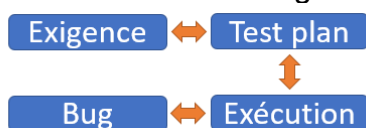
Burn-down charts : Un graphique partagé qui représente l'effort réalisé par rapport au temps



Plan de test Maître : doc qui contient le **périmètre** de test, les approches de test à adopter, les niveaux de test, les critères d'entrée et les critères de sortie de chaque niveau de test, les types de test, le planning des tests, l'organisation du projet avec les rôles, les responsabilités, les réunions, leurs buts et leurs fréquences, les environnements de test et les outils de test.

GCL : gestion de config logiciel (contient toutes les versions précédentes stable du logiciel ainsi que les outils de test, testware.. pour éviter l'incohérence)

Traçabilité bidirectionnelle : chaque exigence est liée aux cas de tests qui la vérifient, et que chaque cas de test est lié aux exigences qu'il couvre



Dette technique : les code mort (code qui n'est jamais exécuter)

Minutes :(Compte Rendu)

Steps =procédures: test manuel ? (to verify)

Script : test automatisé

! Partie d'équivalence : toutes les parties valides et invalides devraient être couvertes

! Compagne de test = manuel suite de test= automatisé

! Les tests automatisés : sont les tests de vie, les tests répétitifs ou le test qu'on est sûr qu'ils ne vont pas changé au cours du temps

! Couverture des branches : vaux mieux prendre une valeur et sa négation (plus facile de tester)

! Prédicat : étape décisionnelle

! Le rôle de responsable de test se termine durant le processus de test à l'étape d'analyse

! Terminer= clôturer les tests : les tests sont dérouler comme prévue de A à Z

! Stopper = arrêter imprévue (nb bug énorme sont majeurs, lenteur énorme, budget ou temps épuisé

! Métrique = quantifiable

Testware : produit d'activité / résultat de chaque phase de processus de test